

Assignment: SQL and R – Movie Ratings

Objective

Collect simple movie-rating data, store it in a SQL database, and analyze it in R.

1. Implementation Notes:

PostgreSQL is the recommended (and supported) relational database for this assignment. If you are unable to install and configure PostgreSQL in your environment, you may find [SQLite](#) easier to use. Students experienced with relational databases may use any local or cloud-based database for their assignment.

2. **NEVER include passwords in your code.** For this assignment, the ability to reproduce your work in my environment is not required, but you need to provide all of the necessary code, include the code used to create and populate tables.

Task Description

1. Select **six recent popular movies** (or television episodes or books or songs or ...)
2. Ask **at least five people** to rate each movie they have seen on a **1–5 scale**.
3. Store the collected ratings in a **SQL database** of your choice.
4. Load the data from SQL into **R** as a dataframe.

Implementation Notes

- The assignment is intentionally open-ended; multiple approaches are acceptable.
- You may:
 - Query the SQL database directly from R, **or**
 - Export SQL data to a `.csv` file and load it into R.
- If you use a GUI to create or populate tables, you must also provide the **SQL code** needed to recreate them.

Reproducibility

- Your solution should be reproducible assuming the same SQL server and R environment.

- You may remove or mask database passwords.
- Full reproducibility of survey responses is *not* required.

Missing Data

- You must demonstrate a **reasonable strategy for handling missing ratings**, since not all participants will have seen all movies.

Optional Enhancements

You may improve your solution by:

- Using survey software to collect ratings
- Managing database credentials securely
- Designing a **normalized database schema**
- Exploring whether **standardizing ratings** is beneficial

Submission Requirements

- Post all code (SQL, R, Quarto) in a **GitHub repository**
- Submit the GitHub link
- RPub's publication is *not required*
- Group work is allowed, but each student must submit individually and list collaborators

Professional Expectation

Treat this as portfolio-quality work suitable for a job interview.

2. Optional Instructions to Make This Easier for Students New to SQL and Recommender Systems

A. Suggested Conceptual Roadmap

Consider approaching the assignment in **three layers**:

1. Data Collection (Problem Framing)

- Think of this as a *mini user-item ratings dataset*.

- Users = people you survey
- Items = movies
- Ratings = integers from 1–5 (with missing values allowed)

2. Data Storage (SQL Fundamentals)

- Start with **two or three tables**, not one:
 - `users(user_id, name)`
 - `movies(movie_id, title)`
 - `ratings(user_id, movie_id, rating)`
- Learn:
 - Primary keys
 - Foreign keys
 - Why many-to-many relationships require a junction table

3. Data Analysis (R Fundamentals)

- Use R primarily for:
 - Data inspection
 - Cleaning missing values
 - Simple summaries (means, counts)
- At this stage, you are *not* building a full recommender—only preparing the data.

B. Concrete Starter Checklist

SQL

- Create tables using `CREATE TABLE`
- Insert data using `INSERT INTO`
- Use `SELECT` with `JOIN` to combine tables

- Verify missing ratings using NULL

R

- Use DBI / RSQLite or RPostgres *or* read a CSV
- Convert SQL output into a tidy dataframe
- Use dplyr to:
 - Count ratings per user and movie
 - Compute average ratings

Recommender Systems (Conceptual Only)

- Understand that this dataset resembles:
 - **A user-item matrix**
 - The raw input for collaborative filtering
- No advanced algorithms are required yet